

Dynamic memory allocation

C programming · CM10 · 90 minutes

ISTIC · University of Rennes 2026-2027



Louis Ledoux

Four integers, then one hundred?

```
size_t count = 4;
int p[4];
for (size_t i = 0; i < count; ++i)
    p[i] = 0;
printf("first=%d, last=%d\n", p[0], p[count - 1]);
```

p has room for **four integers**. What changes when the user asks for **100**?

Live example: [demos/00-allocation/main.c](#)

Reading a count does not enlarge the array

```
size_t count;
int p[4];
if (scanf("%zu", &count) != 1) return EXIT_FAILURE;
if (count == 0 || count > 4) {
    fputs("Choose 1 to 4: the array has four elements.\n", stderr);
    return EXIT_FAILURE;
}
```

Input **4** works. Input **100** reaches the capacity check.

Live example: [demos/00-allocation/checkpoints/02-capacity.c](https://demos.00-allocation/checkpoints/02-capacity.c)

Storage for 100 integers

```
size_t count = 100;
int *p = malloc(count * sizeof *p);
if (p == NULL) {
    fputs("Allocation failed\n", stderr);
    return EXIT_FAILURE;
}
```

malloc returns the block's address, or **NULL** on failure. The contents are **uninitialised**.

Live example: [demos/00-allocation/checkpoints/03-heap.c](#)

Initialise, then use the block

```
for (size_t i = 0; i < count; ++i)
    p[i] = 0;
printf("first=%d, last=%d\n", p[0], p[count - 1]);
```

One contiguous allocation

Valid indices are **$0 \leq i < \text{count}$** . Write each value before reading it.

Live example: `demos/00-allocation/checkpoints/03-heap.c`

Finish the job with free

```
void free(void *pointer);    // From <stdlib.h>  
free(p);                    // After the last use.  
p = NULL;
```

End the allocation's lifetime

Pass the start address of a live allocation from malloc, calloc or realloc. free returns no value.

free(NULL) has no effect. Free each allocation **once**.

Invalid or repeated free is **undefined behaviour**.

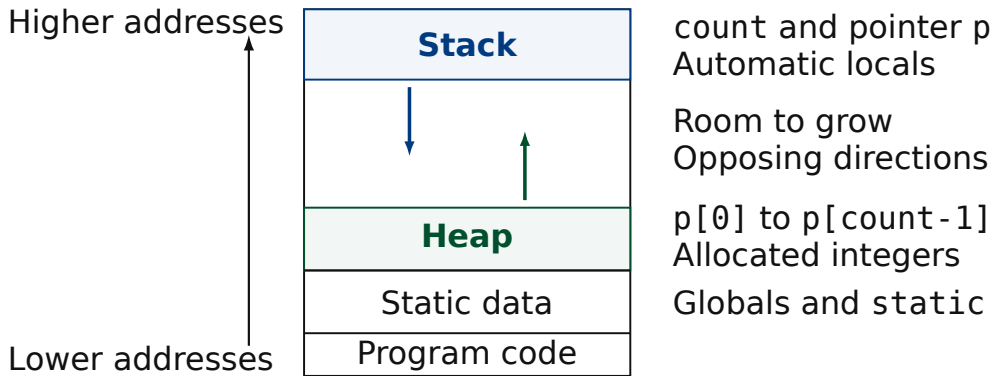
Count elements, request bytes

```
void *malloc(size_t size);    // From <stdlib.h>  
int *p = malloc(100 * sizeof *p);
```

- ▶ `size_t` is unsigned, with at least **16 value bits**.
- ▶ `sizeof` is an **operator** giving a size in bytes.
- ▶ `sizeof *p` is the size of one `int`, not the pointer.

Sizes depend on the platform. In C, the `void *` result needs no cast.

Where does the program keep its data?



A typical layout, not a C language guarantee. Not to scale.

Who controls an object's lifetime?

Static storage

Globals and static objects exist for the **whole program**. Storage is reserved before execution.

Automatic storage

An ordinary local exists during its **block execution**. Each recursive call gets its own locals.

The implementation manages their storage. Deep recursion can **exhaust the stack**.

Live example: `demos/14-memory-layout/main.c`

Four functions in `<stdlib.h>`

Function	Purpose
<code>malloc(bytes)</code>	Allocate uninitialised storage
<code>calloc(count, size)</code>	Allocate storage with all bits zero
<code>realloc(p, bytes)</code>	Resize an allocation; it may move
<code>free(p)</code>	Release an allocation

Compile, run, then inspect

```
gcc -std=c17 -Wall -Wextra -g -O0 main.c -o program  
./program  
valgrind --leak-check=full ./program
```

Read the **error summary**, then the **heap summary**. A program can print the expected result and still misuse memory.

Live example: `demos/00-allocation/checkpoints/03-heap.c`

One character creates an overrun

```
for (size_t i = 0; i <= count; ++i)
    p[i] = 0;
printf("first=%d, last=%d\n", p[0], p[count - 1]);
```

Which iteration writes outside the allocation?

Live example: [demos/00-allocation/checkpoints/04-overrun.c](#)

Allocated does not mean initialised

```
printf("first=%d, last=%d\n", p[0], p[count - 1]);
```

The initialization loop has been removed. What can the reads produce?

```
valgrind --track-origins=yes ./program
```

Live example: [demos/00-allocation/checkpoints/05-uninitialised.c](#)

Memory leak: lose the address

`p = NULL; // No free before the only address is lost.`

Pointer



Allocated block



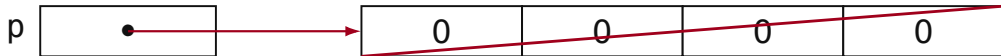
Address lost; block still allocated

Live example: `demos/00-allocation/checkpoints/06-leak.c`

Dangling pointer: the object is gone

Pointer

Allocated block



Freed storage

```
free(p);  
printf("first=%d, last=%d\n", p[0], p[count - 1]);
```

Live example: [demos/00-allocation/checkpoints/07-dangling.c](#)

Local objects can expire too

```
int *bad_result(void) {  
    int value = 42;  
    return &value;  // The local dies when this call ends.  
}
```

A dangling pointer need not come from free

Returning an address does not extend an automatic object's lifetime.

Who is responsible for reclamation?

C: explicit responsibility

For every successful allocation, decide who will release it and when.
Repeated leaks consume more memory.

Java: garbage collection

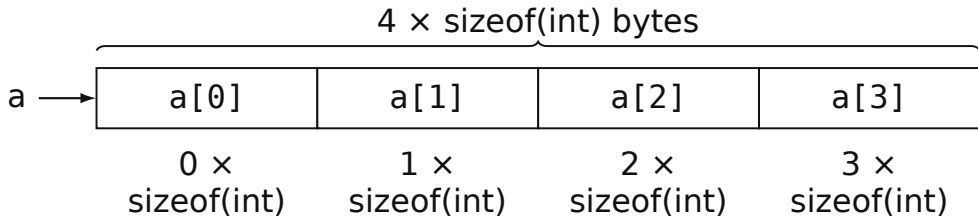
The runtime can reclaim unreachable objects automatically.

Checkpoint · questions 1-5

Answer questions **1-5**, then discuss your reasoning.
Open the allocation quiz.

A one-dimensional integer array

```
int *a = malloc(4 * sizeof *a);
```

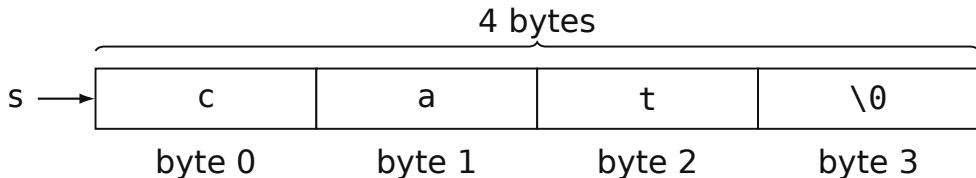


Use $a[i]$, equivalently $*(a + i)$, for $0 \leq i < 4$.

Live example: `demos/04-array/main.c`

A character array can hold a string

```
char *s = malloc(4 * sizeof *s);
```



Three letters need **four bytes**, including '`\0`'.

Live example: [demos/05-string/main.c](#)

Assigning a pointer is not copying text

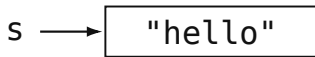
```
char *s = malloc(100);  
if (!s) return EXIT_FAILURE;  
s = "hello";  
char *p = s;  
s = "bye";
```

Predict the values of `p` and `s`, and the fate of the allocation.

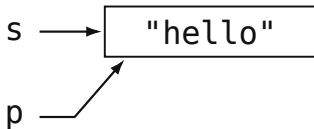
Live example: `demos/03-leak/main.c`

Follow the three assignments

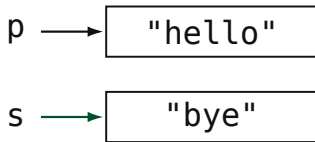
`s = "hello"`



`p = s`



`s = "bye"`



Original heap block: 100 bytes, now unreachable

The literals are separate from the lost heap block. Neither pointer can now be passed to `free`.

Copy characters into owned storage

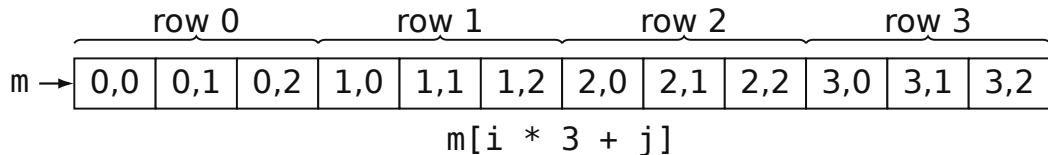
```
char *s = malloc(6);  
if (!s) return EXIT_FAILURE;  
strcpy(s, "hello"); // Include <string.h>.  
printf("%s\n", s);  
free(s);
```

Alternatively, just reference a literal

Use `const char *s = "hello";` when no writable copy is needed.

A 2D array in one block

```
int *m = malloc(4 * 3 * sizeof *m);  
/* After checking m: element (i, j) is m[i * 3 + j]. */
```

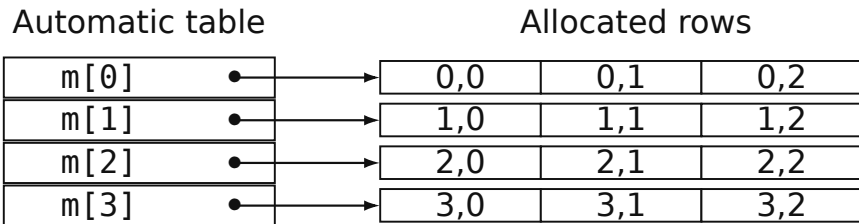


One allocation, one **free(m)**.

Live example: demos/06-matrix-flat/main.c

An array of row pointers

```
int *m[4]; // Automatic table of four pointers.  
for (size_t i = 0; i < 4; ++i)  
    m[i] = malloc(3 * sizeof *m[i]);
```

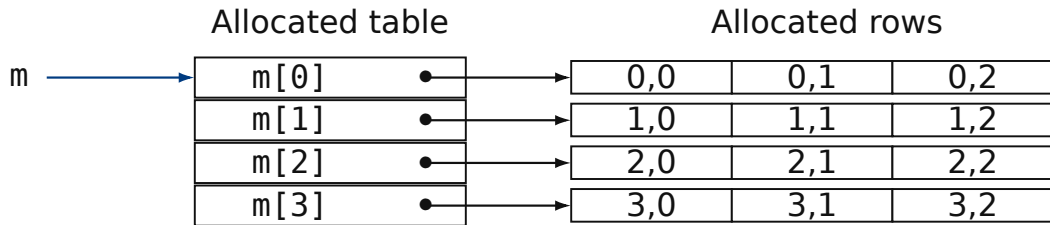


Check each row. Use `m[i][j]`; free each row, **not** `m`.

Live example: [demos/07-matrix-rows/main.c](#)

Allocate the pointer table too

```
int **m = malloc(4 * sizeof *m);  
for (size_t i = 0; i < 4; ++i)  
    m[i] = malloc(3 * sizeof *m[i]);
```



Check `m` first, then each row. Free **rows first**, then `free(m)`.

Live example: [demos/08-matrix-dynamic/main.c](#)

Checkpoint · questions 6-8

Answer questions **6-8**, then discuss your reasoning.
Open the allocation quiz.

Three tools

`enum` names integer constants. `struct` groups members. `typedef` introduces a name for a type.

An address has a street and a number. How does changing a member affect the object?

Live example: `demos/19-types/main.c`

Enumerations

```
enum day { mon, tue, wed, thu, fri, sat, sun };  
enum day today;  
today = wed;
```

By default, values start at **0** and increase by **1**.

Named integer constants

mon is 0; wed is 2; sun is 6.

A structure groups several members

```
struct address {  
    char street[100];  
    int number;  
};
```

A type definition

Members may be integers, floating-point values, arrays, pointers or other structures. No address object exists yet.

Define a type, then create objects

```
struct address home;  
struct address office;
```

These definitions allocate **two distinct objects** of the same type.

Local objects need initialisation

Ordinary local members have indeterminate values until initialised or assigned. The type definition alone sets no values.

Initialising a structure

```
struct address home = { "Paul Bert", 12 };
```

Or combine the type and object definitions:

```
struct address {  
    char street[100];  
    int number;  
} home = { "Paul Bert", 12 };
```

Padding

The implementation may insert unused bytes between members and at the end to satisfy alignment requirements.

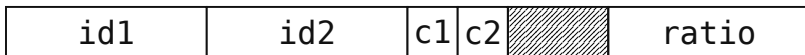
- ▶ Members appear in declaration order.
- ▶ `sizeof(struct T)` includes padding.
- ▶ Layout and alignment depend on the implementation.

Same members, different order

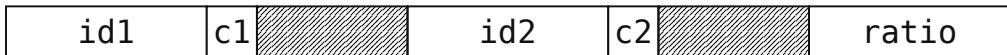
```
struct grouped {  
    int id1, id2;  
    char c1, c2;  
    float ratio;  
};  
struct interleaved {  
    int id1; char c1;  
    int id2; char c2;  
    float ratio;  
};
```

See where the padding goes

Grouped: 16 bytes



Interleaved: 20 bytes



Here: 4-byte int/float, alignment 4; 1-byte char.

16 vs 20 bytes in this layout. Measure with `sizeof` and `offsetof`.

Live example: `demos/09-struct-padding/main.c`

Access members through an object

```
struct address home = { "Paul Bert", 12 };  
strcpy(home.street, "Rue de Paris");  
home.number = 14;
```

The dot operator

Use `.` with a structure object.

`strcpy` needs `<string.h>` and enough space for the terminator.

Access members through a pointer

```
struct address *p = &home;  
strcpy(p->street, "Rue de Paris");  
p->number = 16;
```

The arrow operator

Use `->` with a pointer. `p->number` is the same member as `(*p).number`.

Both forms above modify **home**.

typedef: a name for a type

```
typedef int LENGTH;  
typedef enum { FALSE, TRUE } BOOLEAN;  
typedef struct date DATE;
```

Alias

A typedef introduces a type name, not an object or a distinct underlying type. Then write `LENGTH width;`.

Combine a date and a day of the week

```
struct date { short day, month; int year; };  
typedef enum { mon, tue, wed, thu, fri, sat, sun } DAY;  
typedef struct { DAY day; struct date date; } A_DATE;
```

A nested structure

A_DATE contains a DAY and a complete struct date.

Access nested members

```
A_DATE today = { wed, { 3, 9, 2014 } };  
today.day = wed;  
today.date.year = 2014;  
A_DATE *p = &today;  
p->day = today.day;  
p->date.year = 2026;
```

`p->date.year` reaches the nested member of **the same object**.

Live example: `demos/10-nested-struct/main.c`

Checkpoint · questions 9-12

Answer questions **9-12**, then discuss your reasoning.
Open the allocation quiz.